

# Command-Line Shell & File Encryption Engine (Classical DES)

สัดส่วนคะแนน: 20% จาก 100% ของรายวิชา

สถาปัตยกรรมเป้าหมาย: x86 assembly (32-bit Protected Mode, MASM Standard)

สภาพแวดล้อม: ใช้ Build Tools จาก Visual Studio หรือ MASM32 และอนุญาตให้ใช้ Irvine32 Library

ขนาดกลุ่ม: นักศึกษาจำนวน 3-4 คนต่อหนึ่งกลุ่ม

## 1. วัตถุประสงค์

วัตถุประสงค์หลักของงานชิ้นนี้คือการบูรณาการแนวคิดการทำงานในระดับต่ำของระบบเพื่อสร้างเป็นโปรแกรมภาษา **assembly** ที่สามารถใช้งานได้จริง โดยผู้เรียนจะได้พัฒนาโปรแกรมที่มีส่วนติดต่อผู้ใช้แบบบรรทัดคำสั่ง (Command Line Interface) คล้าย Shell ของระบบปฏิบัติการ ซึ่งโปรแกรมสามารถอ่านไฟล์จาก disk วิเคราะห์คำสั่งโดยใช้ Finite State Machine (FSM) และสามารถเข้ารหัสลับและถอดรหัสลับข้อมูล และนำข้อมูลที่อยู่ในบัฟเฟอร์มาแสดงผลเป็นเลขฐาน 16 ได้

## 2. กฎเกณฑ์และข้อจำกัดเชิงเทคนิค

- การควบคุมทิศทางการทำงานในระดับต่ำ: ห้ามใช้ **directives** การควบคุมทิศทางการทำงานระดับสูงที่ MASM มีให้ (กล่าวคือห้ามใช้ **.IF**, **.WHILE**, **.REPEAT**) ภายใน **code** บริเวณ **loop** ที่ใช้เข้ารหัสลับและถอดรหัสลับ ซึ่งเป็นส่วนที่ประสิทธิภาพมีความสำคัญมาก ผู้เรียนต้องใช้ **instructions** พื้นฐานสำหรับเปรียบเทียบ (**CMP**, **TEST**) และ **conditional jump** (**JE**, **JNE**, **JB**, etc) รวมถึงการอุปในการเข้ารหัสลับและถอดรหัสลับ
- มาตรฐานการเรียก **procedures**: การเรียก **procedures** ต้องทำตามแนวปฏิบัติมาตรฐาน **stack frame pointer** (**push ebp / mov ebp, esp / pop ebp**) โดยทุก **procedures** ต้องเก็บค่า **registers** ของ **caller** ได้ และค่า **return** ที่ส่งคืน **caller** ต้องส่งคืนผ่านค่าของ **EAX**

- c) ห้ามใช้ **External encryption library**: การประมวลผลด้านการเข้ารหัสลับและถอดรหัสลับทั้งหมด (initial Permutation, Key Schedule, 16 Feistel Rounds, S-Box lookups, Inverse-Permutation) จะต้องถูกเขียนเองด้วย x86 assembly
- d) ขอบเขตของ **File I/O**: การดำเนินการที่เกี่ยวข้องกับไฟล์จะต้องใช้ฟังก์ชันจาก Win32 API (CreateFileA, ReadFile, WriteFile, CloseHandle) หรือ procedures ที่ดำเนินการกับไฟล์ จาก Irvine32 library (OpenInputFile, ReadFromFile, WriteToFile, CloseFile)

### 3. Work Breakdown Structure

#### Module A: Shell Core & FSM Command Parser

ความรับผิดชอบ:

- อุปกรณ์การทำงานหลัก ซึ่งอ่านบรรทัดคำสั่งจากหน้า console เข้าไปยัง input buffer
- สร้าง Finite State Machine (FSM) เพื่อนำ string ที่ได้รับมาวิเคราะห์คำ (parse) ตามคำสั่ง (KEYGEN, ENCRYPT, DECRYPT, DUMP, STATS, CLEAR, EXIT) รวมถึงนำชื่อไฟล์และ key ที่ใช้เข้ารหัสลับหรือถอดรหัสลับมาวิเคราะห์คำ (parse) โดยโปรแกรมจะต้องรองรับคำสั่งดังต่อไปนี้
  - 1) KEYGEN: รับค่า key ขนาด 64 bits แสดงผล key schedule ทั้ง 16 keys ( $K_1 \dots K_{16}$ )
  - 2) ENCRYPT: รับค่าชื่อไฟล์ที่เก็บ plaintext และ key จากนั้นสร้างไฟล์ที่เข้ารหัสลับข้อมูล
  - 3) DECRYPT: รับค่าชื่อไฟล์ที่เก็บ ciphertext และ key จากนั้นสร้างไฟล์ที่ถอดรหัสลับข้อมูลแล้ว
  - 4) DUMP: รับค่าชื่อไฟล์ plaintext หรือ ciphertext จากนั้นแสดงผลข้อมูลในไฟล์ทั้งในรูปแบบเลขฐาน 16 และรูปแบบ ASCII พร้อม offset
  - 5) STATS: รับค่าชื่อไฟล์ plaintext หรือ ciphertext จากนั้นแสดงผลข้อมูล histogram ของ byte patterns ที่พบในไฟล์
  - 6) CLEAR: ไม่รับค่าใดเพิ่มเติม โดยเป็นคำสั่งที่ล้างผลลัพธ์บนหน้าจอแล้ววนกลับมาขอ input ถัดไป
  - 7) EXIT: ไม่รับค่าใดเพิ่ม โดยเป็นคำสั่งที่จบการทำงานของโปรแกรม
- ตรวจสอบความถูกต้องของ input ตามไวยากรณ์ที่กำหนด และแสดงข้อความแจ้งข้อผิดพลาดสำหรับ input ที่ไม่ต้องถูก

## Module B: DES Key Schedule Generator

ความรับผิดชอบ:

- รับ key ขนาด 64 bits ตามมาตรฐาน (effective key ขนาด 56 bits และ parity อีก 8 bits)
- ทำการสลับบิตแบบ Permuted Choice 1 ( $PC-1$ ) และ Permuted Choice 2 ( $PC-2$ ) โดยใช้การ mask และ shift (AND, SHL, SHR, OR)
- ประมวลผลตามตรรกะที่กำหนดจำนวน 16 รอบ โดยดำเนินการ left circular rotations (ROL) 1-bit และ 2-bit ทั้งกับครึ่ง 28-bit แรกและครึ่ง 28-bit หลัง ( $C_i, D_i$ ) เพื่อสร้าง key ย่อยขนาด 48 bits ( $K_1 \dots K_{16}$ )

## Module C: DES 16-Round Feistel Core Engine

ความรับผิดชอบ

- ทำ Initial Permutation ( $IP$ ) และ Inverse Initial Permutation ( $IP^{-1}$ ) กับบล็อกข้อมูลขนาด 64 bits โดยใช้ register ขนาด 32 bits จำนวน 2 ตัว (คู่ EDX:EAX หรือใช้ memory buffer)
- ทำฟังก์ชัน Feistel round  $f(R_{i-1}, K_i)$ :
  - 1) Expansion ( $E$ ): ขยายครึ่งหนึ่งที่มีขนาด 32 bits ไปเป็น 48 bits.
  - 2) Subkey XOR: ทำการ XOR กับ subkey  $K_i$  ขนาด 48 bits
  - 3) S-Box Lookups: แบ่งข้อมูลขนาด 48 bits เป็น 8 บล็อก บล็อกละ 6 bits โดยใช้ 2D lookup array ที่กำหนดไว้ใน data segment (.data) เพื่อทำแยกบิตที่ต้องการ (bit extraction)
  - 4) Permutation ( $P$ ): สลับลำดับ output จาก S-box ซึ่งเป็นข้อมูลขนาด 32 bit
- วนลูป Feistel 16 รอบ (วนลูปลำดับไปข้างหน้าสำหรับการเข้ารหัสลับ และวนลูปลำดับ subkey ย้อนกลับสำหรับการถอดรหัสลับ) ในโหมดการทำงาน Electronic Codebook (ECB) และให้เติม PKCS#7 padding ในไฟล์ด้วย

## Module D: Memory Dumper & Buffer Analytics

ความรับผิดชอบ

- ทำ procedure ชื่อ DisplayHexDump: รับ memory offset pointer และขนาดความยาวข้อมูลในหน่วย byte เพื่อแสดงผลข้อมูลที่ละ 16 bytes ต่อบรรทัด โดยให้แสดง offset address

01476105 Computer Organization and Assembly Language  
Computer Engineering and Cyber Security, KMITL

พร้อมค่าทั้งในรูปแบบเลขฐาน 16 และแบบ ASCII ในกรณีที่ค่าเป็น non-printable character (ค่าอยู่ในช่วงน้อยกว่า 20h หรือมากกว่า 7Eh) ให้แสดงผลเป็นจุด (.)

- ทำ procedure ชื่อ ComputeBufferStats: รับ input buffer และ output buffer เพื่อสร้าง histogram แจกแจงความถี่ของ byte pattern ก่อนและหลังการเข้ารหัสลับ โดยมีจำนวน bin เป็น 256
- แสดง histogram ค่า ASCII ที่มีความถี่สูงสุดเพื่อแสดงคุณสมบัติการกระจาย/การสุ่มของ DES

## 4. เกณฑ์การประเมิน

| Module | เงื่อนไขการตรวจสอบ                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | คะแนน |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| A      | <ul style="list-style-type: none"><li>- รันไดบน Windows 10 หรือ 11</li><li>- วนรับคำสั่งแบบ REPL จนกว่าจะพิมพ์คำสั่ง EXIT</li><li>- สามารถ handle บรรทัดคำสั่งที่ถูกต้องได้ตามที่กำหนด (ENCRYPT, DECRYPT, DUMP, KEYGEN, STATS, CLEAR, EXIT)</li><li>- สามารถตรวจสอบบรรทัดคำสั่งที่ไม่ถูกต้องหรือมีปัญหา โดยโปรแกรมยังทำงานต่อไปได้</li><li>- สามารถแจ้ง error message ที่เหมาะสมเมื่อบรรทัดคำสั่งที่เป็น input ผิดหรือมีปัญหา</li><li>- สามารถอ่านค่าจากไฟล์หรือเขียนผลลัพธ์ลงไฟล์ได้ถูกต้องเหมาะสม</li></ul> | 6     |
| B      | <ul style="list-style-type: none"><li>- สามารถสร้าง key ที่ใช้เพื่อเข้ารหัสลับและถอดรหัสลับได้</li><li>- Key ที่สร้างขึ้นตรงตามมาตรฐาน DES</li></ul>                                                                                                                                                                                                                                                                                                                                                          | 4     |
| C      | <ul style="list-style-type: none"><li>- เข้ารหัสลับข้อมูลได้ถูกต้อง</li><li>- ถอดรหัสลับข้อมูลได้ถูกต้อง</li><li>- ประมวลผลในโหมด ECB และได้ PKCS#7 padding ได้อย่างถูกต้อง</li></ul>                                                                                                                                                                                                                                                                                                                         | 6     |
| D      | <ul style="list-style-type: none"><li>- แสดงผลจากการใช้คำสั่ง Dump ตามรูปแบบที่กำหนด</li><li>- แสดงผล non-printable character เป็นจุดได้ตามที่กำหนด</li><li>- ประมวลผล Histogram เป็น 256 bins ได้ถูกต้อง</li><li>- สามารถแสดงค่า top occurrences ของ byte pattern ที่ปรากฏได้ถูกต้อง</li></ul>                                                                                                                                                                                                               | 4     |

## 5. กฎเกณฑ์การหักคะแนน

| เงื่อนไข                                                                            | คะแนนที่ถูกหัก                                                                   |
|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| มีการใช้ Directives ที่ห้าม (.IF, .WHILE, หรือ .REPEAT) ใน cipher หรือ parser loops | -2 คะแนนต่อจำนวนครั้งที่ปรากฏการใช้งาน โดยหักคะแนนตามเงื่อนไขนี้ไม่เกิน -6 คะแนน |
| โปรแกรม crash แบบไม่คาดคิด                                                          | -1 คะแนนต่อสาเหตุที่ทำให้ crash โดยหักคะแนนตามเงื่อนไขนี้ไม่เกิน -4 คะแนน        |
| มีการเรียกใช้ External Library สำหรับประมวลผล key หรือการเข้ารหัสลับ                | -2 คะแนนต่อจำนวนครั้งที่ปรากฏการใช้งาน โดยหักคะแนนตามเงื่อนไขนี้ไม่เกิน -6 คะแนน |
| ส่งช้ากว่าที่กำหนด                                                                  | -2 คะแนนต่อวัน โดยสูงสุดไม่เกิน 5 วัน                                            |
| อธิบายหรือตอบคำถามเกี่ยวกับงานของตนเองไม่ได้                                        | -1 คะแนนต่อคำถาม หักตามเงื่อนไขนี้ไม่เกิน -5 คะแนน                               |

## 6. การส่งงาน

- ให้ไฟล์ **readme.txt** ภายในไฟล์ระบุชื่อและรหัสนักศึกษาของสมาชิกในกลุ่ม พร้อมทั้งระบุความรับผิดชอบของสมาชิกในกลุ่มแต่ละคนเป็นผู้พัฒนา **code** ส่วนใดบ้าง
- ให้ส่ง **source code** ภาษา **assembly** พร้อมไฟล์ **readme.txt** ผ่านระบบ **Assignment** บน **Microsoft Teams** โดยให้ตัวแทนกลุ่ม 1 คนเป็นผู้ส่ง
- เมื่อนำส่งไฟล์แล้ว ให้นัดเวลากับผู้สอน เพื่อ **demo** งาน โดยใช้เวลาในช่วงคาบปฏิบัติได้ และมีเงื่อนไขว่าสมาชิกทุกคนต้องมาส่งงานพร้อมกันเท่านั้น
- หากมีข้อกำหนดของโปรแกรมที่ไม่ชัดเจน หรือมีส่วนแสดงผลใดๆ ที่อยู่นอกเหนือจากตัวอย่างการรัน อนุญาตให้ผู้เรียนสามารถเพิ่มเติมหรือปรับเปลี่ยนได้ตามความเหมาะสม โดยยังคงความถูกต้องในการเข้ารหัสลับและถอดรหัสลับข้อมูล

## 7. Sample Run

### Test Vectors

เพื่อให้มั่นใจว่าอัลกอริทึมเป็นไปตามข้อกำหนด กลุ่มผู้เรียนต้องทดสอบโปรแกรมของตนโดยใช้ **test vector** อย่างเป็นทางการจาก **NIST DES**:

- **Plaintext:** 0123456789ABCDEF (Hexadecimal, 64-bit)
- **Key:** 133457799BBCDFF1 (Hexadecimal, 64-bit)
- **Expected Ciphertext:** 85E813540F0AB405 (Hexadecimal, 64-bit)

### Interactive Shell Run Execution Trace

```
DES-SHELL> DUMP "secret.txt"
[Address] 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F | ASCII
-----
00000000 01 23 45 67 89 AB CD EF 01 23 45 67 89 AB CD EF | .#Eg.....#Eg...

DES-SHELL> ENCRYPT "secret.txt" 0x133457799BBCDFF1
Loading secret.txt (16 bytes)...
Executing DES 16-round key generation...
Processing 2 block(s) in ECB mode...
File encrypted successfully -> "secret.txt.enc"

DES-SHELL> DUMP "secret.txt.enc"
[Address] 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F | ASCII
-----
00000000 85 E8 13 54 0F 0A B4 05 85 E8 13 54 0F 0A B4 05 | ...T.....T....

DES-SHELL> STATS "secret.txt.enc"
Total File Size: 16 Bytes
Entropy Statistics: High Diffusion (Ciphertext Uniformity Check PASSED)
Top Byte Occurrences:
1. [0x85] : 2 occurrences  [**]
2. [0xE8] : 2 occurrences  [**]
3. [0x13] : 2 occurrences  [**]
4. [0x54] : 2 occurrences  [**]
5. [0x0F] : 2 occurrences  [**]

DES-SHELL> EXIT
Exiting DES Command-Line Shell...
```